Ring Buffer Semantics

This note typesets the executable Redex model in `ring.rkt`. A ring state is written as `(ring capacity produced consumed)`. The cursors are monotonic; occupancy is derived by subtracting the consumed cursor from the produced cursor.

State language. The model keeps the ring state deliberately small: capacity, producer cursor, and consumer cursor.

$$
\begin{aligned}
r &::= (\mathsf{ring}\ n\ n\ n) \\
n &::= \mathit{natural}
\end{aligned}
$$

Derived quantities and invariant. The `fill` and `free` metafunctions compute occupancy and available capacity; `valid?` states `0 <= fill <= capacity`.

$$
\begin{aligned}
&\mathsf{fill} : r \to n \\
&\mathsf{fill}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket = n_b\ \text{-}\ n_c \\
&\mathsf{free} : r \to n \\
&\mathsf{free}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket = n_a\ \text{-}\ \mathsf{fill}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket \\
&\mathsf{valid?} : r \to \mathit{boolean} \\
&\mathsf{valid?}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket = n_c <= n_b \text{ and } \mathsf{fill}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket <= n_a
\end{aligned}
$$

Transitions. Producing advances the producer cursor when free space remains. Consuming advances the consumer cursor when the ring is non-empty.

$$
\begin{aligned}
&(\mathsf{ring}\ n_a\ n_b\ n_c) \longrightarrow (\mathsf{ring}\ n_a\ 1 + n_b\ \ n_c) \quad [\mathsf{produce}] \\
&\quad \text{where } \mathsf{free}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket > 0 \\
&(\mathsf{ring}\ n_a\ n_b\ n_c) \longrightarrow (\mathsf{ring}\ n_a\ n_b\ 1 + n_c) \quad [\mathsf{consume}] \\
&\quad \text{where } \mathsf{fill}\llbracket(\mathsf{ring}\ n_a\ n_b\ n_c)\rrbracket > 0
\end{aligned}
$$

Executable check. The model test runs `redex-check` for 5000 attempts, searching for a one-step counterexample to validity preservation.